

AIMS System - Detailed Class Design Documentation

1. Design Class: PaymentTransaction

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	transactionId	int	null	Unique identifier for the transaction
2	amount	float	0.0	Payment amount
3	paymentMethod	String	"Credit Card"	Method of payment
4	transactionDate	Date	current date	Date of transaction
5	status	String	"Pending"	Transaction status (Pending, Success, Failed, Refunded)
6	content	String	null	Transaction details or description
7	orderId	int	null	ID of the associated order

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	evalutePaymentResult	boolean	Processes the payment result from VNPay
2	save	void	Saves the transaction to the database
3	transactionInfo	String	Returns formatted transaction information
4	saveRefundTransaction	void	Saves refund transaction information
5	createTransaction	void	Creates (stores) a new payment transaction in the system

Parameter:

- transactionInfo(transactionId: String) : PaymentTransaction
 - transactionId: An integer representing the unique ID of the transaction.
 - Return: A PaymentTransaction object containing the transaction details.

Exception:

- TransactionNotFoundException if referencing a non-existent transaction.
- InvalidTransactionAmountException – If the amount is invalid (e.g., negative or zero when it shouldn't be).

Method:

- save()
 - Purpose: Commits the current transaction state to persistent storage.
 - Usage: Called after creating or updating any transaction details (e.g., setting the status to “completed”).
- transactionInfo(transactionId: String): PaymentTransaction
 - Purpose: Retrieves the details of a specific payment transaction by its unique ID.
 - Usage: If transactionId exists, returns a PaymentTransaction object. If it does not exist, raises TransactionNotFoundException.

State:

- If transactionId is invalid → TransactionNotFoundException is raised.
- If transactionStatus is "completed" → Payment is successful and no further changes unless a refund is requested.
- If processRefund is successful → transactionStatus might be changed to “refunded”, and isRefunded is set to true.
- refundTransactionId may be generated or updated when a refund is initiated.

2. Design Class: Invoice

Table 1. Attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

1	invoiceId	int	null	Unique identifier for the invoice
2	order	Order	null	Order associated with this invoice
3	issueDate	Date	current date	Date when invoice was created
4	productList	List<OrderItem>	empty list	List of products in the invoice
5	subtotal	float	0.0	Total price excluding VAT
6	VAT	float	0.0	Value Added Tax amount (10% of subtotal)
7	totalWithVAT	float	0.0	Subtotal + VAT
8	regularDeliveryFee	float	0.0	Fee for regular delivery items
9	rushDeliveryFee	float	0.0	Fee for rush delivery items
10	totalAmount	float	0.0	Final amount to be paid
11	paymentMethod	String	"Credit Card"	Method of payment
12	recipientInfo	DeliveryInfo	null	Delivery information of the recipient
13	VAT_RATE	float	0.1	Value-added tax rate (constant 10%)

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	createInvoice	Invoice	Creates a new invoice with order and delivery information
2	calculateSubtotal	float	Calculates the subtotal price of all products
3	calculateVAT	float	Calculates the VAT amount based on subtotal
4	calculateTotalWithVAT	float	Calculates the total amount including VAT
5	calculateDeliveryFees	void	Calculates separate regular and rush delivery fees
6	calculateTotalAmount	float	Calculates the final total amount to be paid
7	generateInvoiceDetails	Map<String, Object>	Generates a complete map of invoice details
8	save	void	Saves the invoice to the system
9	getInvoiceById	Invoice	Retrieves an invoice by its ID
10	Invoice	void	Constructor for creating an invoice

Parameter:

- **createInvoice(cart: Cart) : Invoice**
 - cart: The user’s cart data.
 - Return: A fully built Invoice with items, total amounts, and an assigned ID.

Exception:

- InvoiceSaveException – Thrown if saving the invoice fails (database error, disk error, etc.).
- InvalidCartException – If cart is empty or invalid when calling createInvoice().

Method:

- Invoice() (constructor)
 - Purpose: Sets up default values (e.g., status = "unpaid").
 - Usage: Called internally or externally to create an invoice instance.
- createInvoice(cart: Cart): Invoice
 - Purpose: Takes a cart object, calculates subtotal, VAT, and total, sets invoice items, then returns an Invoice.
 - Usage: Typically called by InvoiceScreen after the user finalizes the cart.
- save()
 - Purpose: Persists the invoice data (items, amounts, status) into a database or storage.

- Usage: Called once the invoice is ready to be stored or updated (e.g., after changing status).

State

- status transitions: “unpaid” → “paid” → possibly “cancelled.”
- totalAmountInclVAT = totalAmountExclVAT * 1.1 (assuming 10% VAT).
- finalAmount = totalAmountInclVAT + deliveryFee.

3. Design Class: VNPay

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	apiKey	String	null	VNPay API key
2	gatewayUrl	String	null	The VNPay endpoint (sandbox or production) for payment requests.

Table 2 . Operation design

#	Name	Return type	Description (purpose)
1	processByVNPay()	void	Processes payments via VNPay
2	processRefund()	boolean	Processes refunds via VNPay

Parameter:

- **ProcessByVNPay(transactionInfo: PaymentTransaction)**
 - transactionInfo: Information required for VNPay payment.
- **processRefund(refundInfo: PaymentTransaction)**
 - refundInfo: Contains details about the refund (order ID, amount, etc.).

Exception

- VNPayConnectionException: If there is a failure to connect or authenticate with VNPay.
- VNPayResponseException: If VNPay returns an error response

State

- If the connection to VNPay fails → VNPayConnectionException.
- If VNPay denies the transaction → VNPayResponseException or returns false.

4. Design Class: PayOrderController

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	currentInvoice	Invoice	null	A reference to the invoice that is being processed for payment.
2	paymentTransaction	PaymentTransaction	null	A reference to the PaymentTransaction created once payment is initiated or completed.

Table 2. Operation Design

#	Name	Return type	Description (purpose)
1	payOrder	boolean	Initiates payment process for an invoice and returns success/failure
2	paymentFailed	void	Handles the scenario when the payment fails (e.g., logs, notifies user)
3	savePaymentResult	boolean	Persists the final payment result (success or failure) into the system

Parameter:

- **payOrder(invoice: Invoice): boolean**
 - **invoice:** The Invoice object to be paid
 - **Return:** true if payment initiation or completion is successful, otherwise false

Exception:

- PaymentProcessingException – Could be thrown if an error occurs during payment (e.g., network failure to VNPay, invalid invoice data, etc.).

- `InvoiceNotFoundException` – If the provided invoice cannot be found or is invalid.

Method:

How to use parameters/attributes:

- **`payOrder(invoice: Invoice): boolean`**
 - **Purpose:** To process payment for the given Invoice. It may call external boundaries (e.g., `VNPayBoundary`) to complete the transaction.
 - **Usage:** Called after the invoice is created and the user chooses payment method. If successful, returns true; on error, might throw `PaymentProcessingException` or return false.
- **`paymentFailed()`**
 - **Purpose:** Triggered when the payment process fails. This method might notify the user interface and log the failure.
 - **Usage:** Called by the system or boundary when a payment gateway reports an error.
- **`savePaymentResult()`**
 - **Purpose:** To store the payment outcome (success, failure, transaction details) into the database or system logs.
 - **Usage:** Called once the payment is confirmed or fails, ensuring the final state is recorded.

State

- If `payOrder` succeeds → The system moves to a “payment completed” state for the Invoice.
- If `payOrder` fails → `paymentFailed()` is invoked; the system may keep the invoice state as “unpaid,” logs the failure, etc.
- `savePaymentResult()` typically changes or finalizes the payment record in the database.